Continuous Delivery

and its design

Yegor Bugayenko

Lecture #10 out of 16
80 minutes

The slidedeck was presented by the author in this YouTube Video.

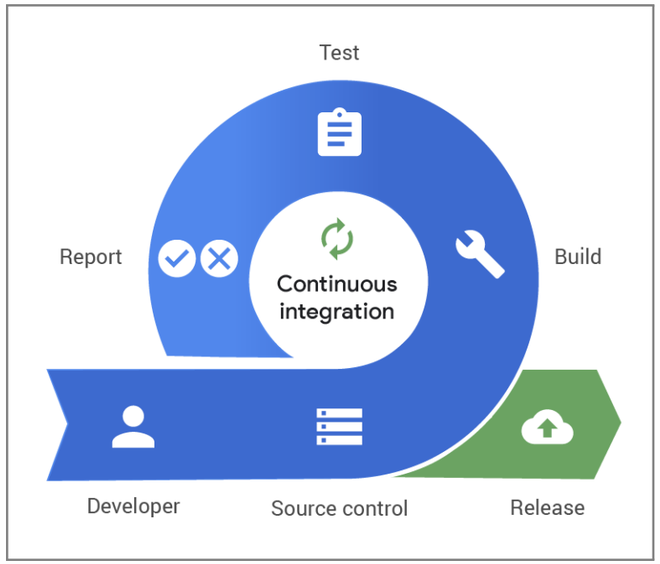
All visual and text materials presented in this slidedeck are either originally made by the author or taken from public Internet sources, such as web sites. Copyright belongs to their respected authors.

- Continuous Integration
- Continuous Delivery
- Open Source in GitHub
- Books, Venues, Call-to-Action

Chapter #1:

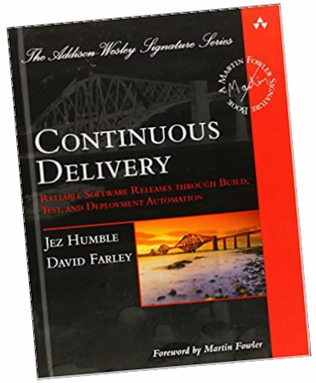
Continuous Integration

How Continuous Integration (CI) works:



It doesn't work :(

[CI Pre-Flight]

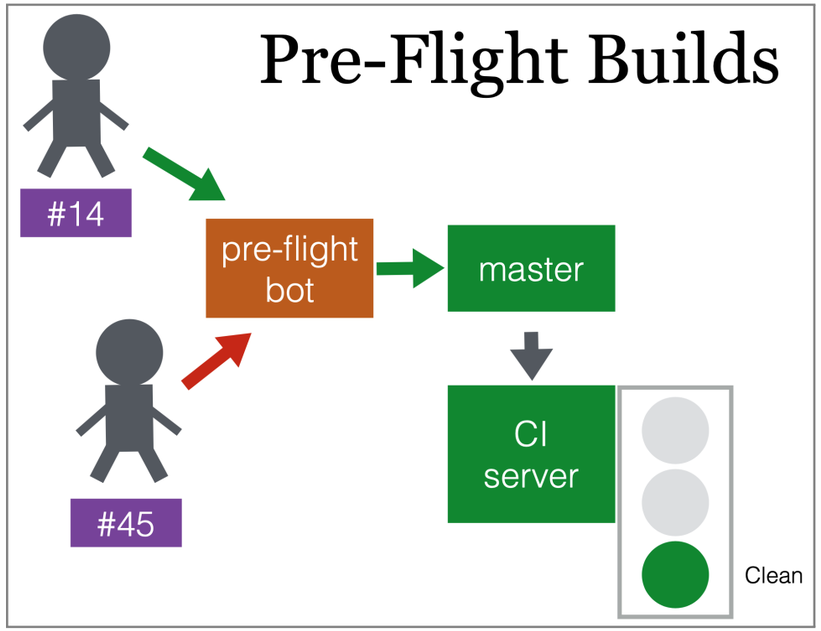




"Crucially, if the build fails, the development team stops whatever they are doing and fixes the problem immediately"

— Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation*. Pearson Education, 2010. DOI: 10.5555/1869904

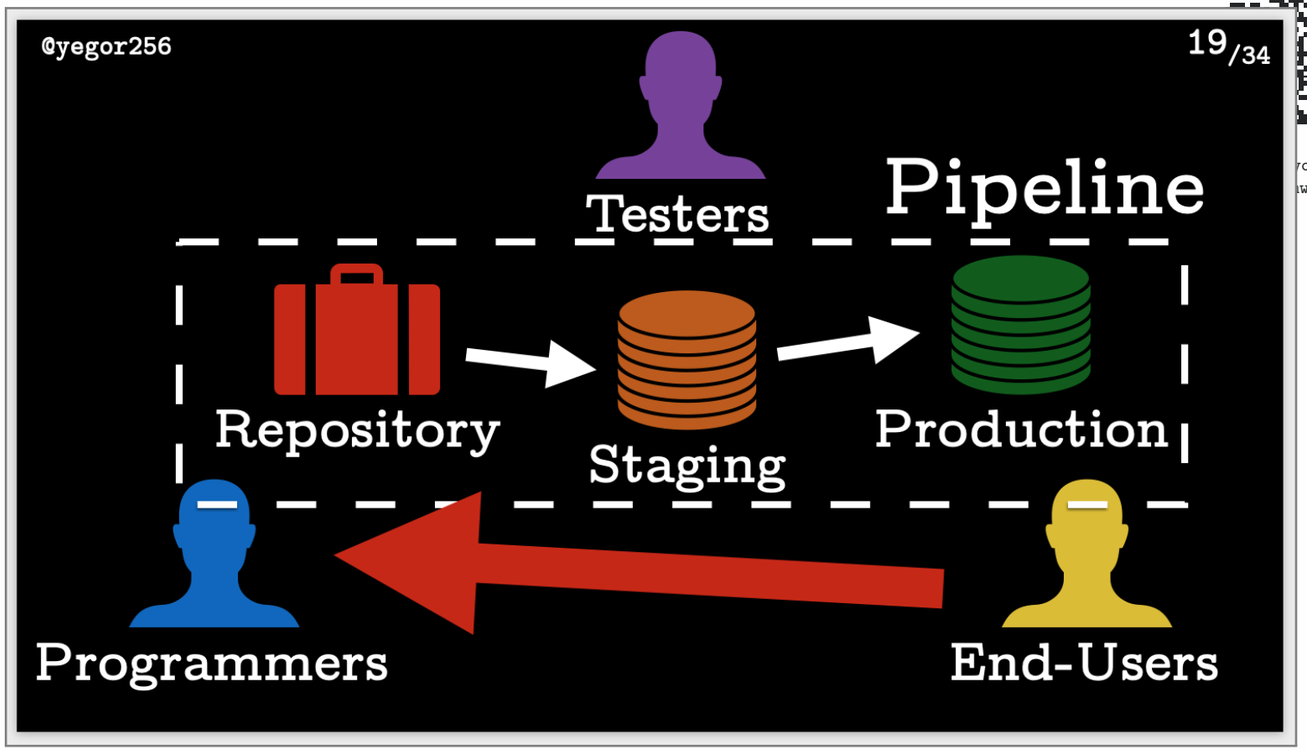
Pre-Flight Builds



Master branch is “read-only”!

Chapter #2:

Continuous Delivery



youtube.com/
wAbuDrV8 →

Joker Conf 2018, Quality vs Quantity, in Russian





"Each stage of a build pipeline is looking for reasons to reject the build. Tests failed? Reject it. Lint complains? Reject it. Build fails integration tests in staging? Reject it. Finished archive smells funny? Reject it."

— Michael Nygard. *Release It!: Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, 2007. DOI: 10.5555/1200767

Quality Wall

Linters

Static analyzers

Unit tests

Integration tests

Test coverage control

Mutation coverage control

Manual code review(s)

Target Platforms

Static web site: GitHub Pages

Library: Maven Central, RubyGems.org, Npm.org, etc.

Small web app: Heroku or Dokku

Bigger web app: AWS Elastic Beanstalk

Mobile app: TestFlight

Versioning

snapshot, alpha, beta, final

semver.org: 1.4.17

0ver.org: 0.43.3

CI/CD Tools

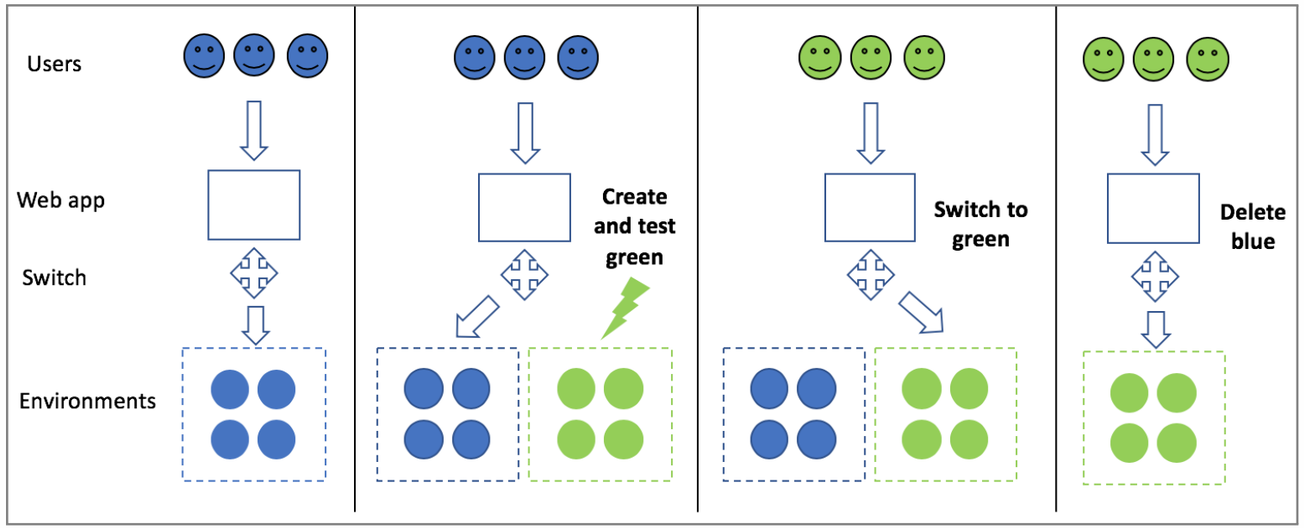
Jenkins

1000+ Hosted Services like Travis

GitHub Actions

Rultor.com

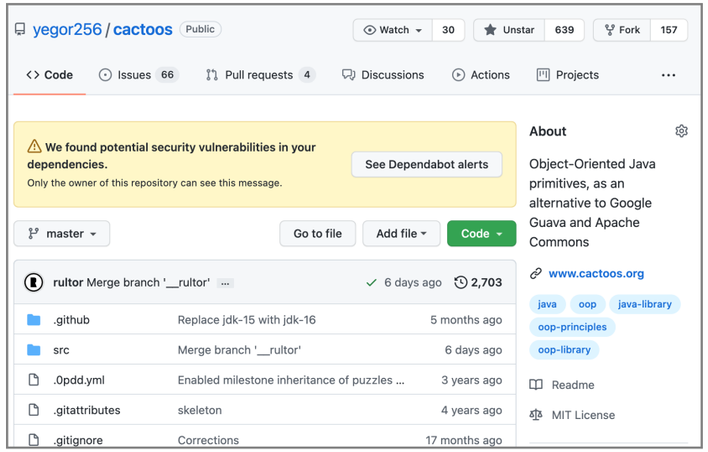
Blue/Green Deployment

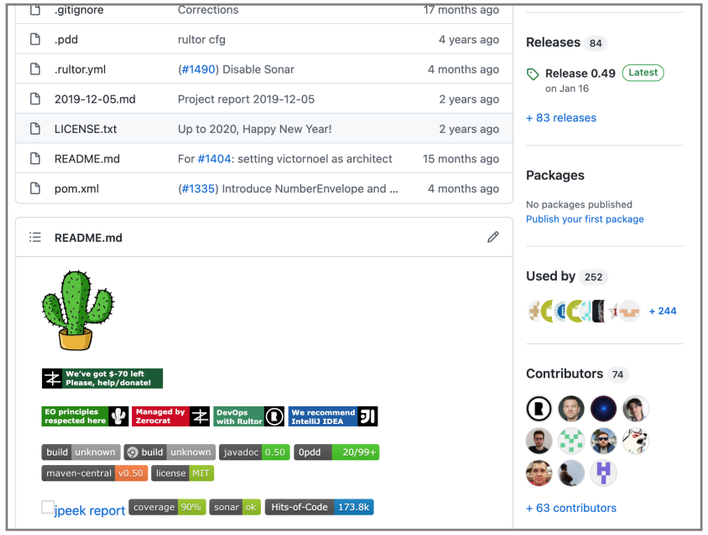


May not work with databases :(

Chapter #3:

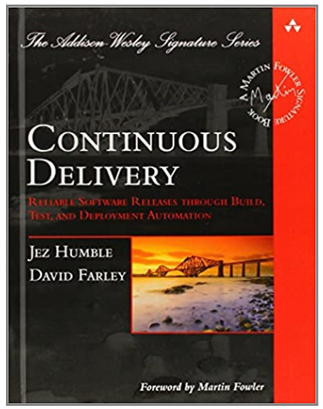
Open Source in GitHub


yegor256 / cactoos
Public
Watch 30
Unstar 639
Fork 157
Code
Issues 66
Pull requests 4
Discussions
Actions
Projects
We found potential security vulnerabilities in your dependencies.
Only the owner of this repository can see this message.
See Dependabot alerts
master
Go to file
Add file
Code
rultor Merge branch '__rultor'
6 days ago
2,703
.github Replace jdk-15 with jdk-16 5 months ago
src Merge branch '__rultor' 6 days ago
.0pdd.yml Enabled milestone inheritance of puzzles ... 3 years ago
.gitattributes skeleton 4 years ago
.gitignore Corrections 17 months ago
About
Object-Oriented Java primitives, as an alternative to Google Guava and Apache Commons
www.cactoos.org
java
oop
java-library
oop-principles
oop-library
Readme
MIT License


.gitignore Corrections 17 months ago
.pdd rultor cfg 4 years ago
.rultor.yml (#1490) Disable Sonar 4 months ago
2019-12-05.md Project report 2019-12-05 2 years ago
LICENSE.txt Up to 2020, Happy New Year! 2 years ago
README.md For #1404: setting victornoel as architect 15 months ago
pom.xml (#1335) Introduce NumberEnvelope and ... 4 months ago
README.md
We've got $-70 left Please, help/donate!
EO principles respected here
Managed by Zerocrat
DevOps with Rultor
We recommend IntelliJ IDEA
build unknown
build unknown
javadoc 0.50
0pdd 20/99+
maven-central v0.50
license MIT
jpeek report
coverage 90%
sonar ok
Hits-of-Code 173.8k
Releases 84
Release 0.49 Latest
on Jan 16
+ 83 releases
Packages
No packages published
Publish your first package
Used by 252
+ 244
Contributors 74
+ 63 contributors

Chapter #4:

Books, Venues, Call-to-Action



Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation*. Pearson Education, 2010. DOI: 10.5555/1869904



Michael Nygard. *Release It!: Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, 2007. DOI: 10.5555/1200767

Where to go:

Collect 1000 stars on GitHub for your product.

Call to Action:

Setup continuous delivery pipeline in your application so that it is released to production automatically on each commit to `master` branch.

Still unresolved issues:

- How to parallelize tests?
- How to speed up merging into “master”?
- How to generate tests automatically?
- How to enforce quality for legacy code base?

References

[1] Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation*. Pearson Education, 2010. DOI: 10.5555/1869904.

[2] Michael Nygard. *Release It!: Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, 2007. DOI: 10.5555/1200767.